Sure Jaswanth Venkata Rahul-192425048

ITA0406-R programming

R Programming - Activity 2

Part A: Data Types in R

1. Define data types in R. Explain the different basic data types with suitable examples.

A data type specifies the kind of value a variable holds and decides which operations can be performed on it. R is dynamically typed, so the type is assigned automatically when a value is stored. R has six basic (atomic) data types:

- **Numeric:** decimal or real numbers, stored as double by default. Example: 45.75
- **Integer:** whole numbers, written with the suffix L. Example: 100L
- **Character:** text values written inside quotes. Example: "R Programming"
- **Logical:** Boolean values TRUE or FALSE. Example: TRUE
- **Complex:** numbers with a real and an imaginary part. Example: 3 + 4i
- **Raw:** values stored as raw bytes. Example: charToRaw("A")

Code:

```
a <- 45.75
b <- 100L
c <- "R Programming"
d <- TRUE
e <- 3 + 4i
f <- charToRaw("A")
class(a)
class(b)
class(c)
class(d)
class(e)
class(f)
```

Output:

```
[1] "numeric"
[1] "integer"
[1] "character"
[1] "logical"
[1] "complex"
[1] "raw"
```

2. Write an R program to create variables of integer, numeric, character, logical, and complex data types. Display their values and identify their types using class().

Code:

```
int_var <- 10L
num_var <- 15.75
char_var <- "Hello R"
logical_var <- TRUE
complex_var <- 2 + 3i
print(int_var)
print(num_var)
print(char_var)
print(logical_var)
print(complex_var)
class(int_var)
class(num_var)
class(char_var)
class(logical_var)
class(complex_var)
```

Output:

```
[1] 10
[1] 15.75
[1] "Hello R"
[1] TRUE
[1] 2+3i
[1] "integer"
[1] "numeric"
[1] "character"
[1] "logical"
[1] "complex"
```

3. Differentiate between integer and numeric data types in R with suitable examples.

- **Integer:** whole numbers only, created with the L suffix, stored in 32 bits, and uses less memory.
- **Numeric:** any real number (whole or decimal). It is the default type for numbers in R and is stored as a double (64 bits).
- **Default behaviour:** typing 5 creates a numeric value, while 5L creates an integer.
- **Division:** dividing two integers with / gives a numeric result.

Code:

```
a <- 5L
b <- 5
c <- 5.5
class(a)
class(b)
class(c)
typeof(a)
typeof(b)
is.integer(a)
is.integer(b)
class(5L / 2L)
```

Output:

```
[1] "integer"
[1] "numeric"
```

```
[1] "numeric"
[1] "integer"
[1] "double"
[1] TRUE
[1] FALSE
[1] "numeric"
```

4. Explain the use of typeof(), class(), and is.numeric() functions in identifying and testing data types in R.

- **typeof():** returns the internal storage type of an object, such as "double", "integer", "character" or "logical".
- **class():** returns the class of an object, which is the type a user normally sees, such as "numeric", "integer", "character" or "data.frame".
- **is.numeric():** tests whether an object is numeric and returns TRUE or FALSE. It returns TRUE for both integer and double values.

Code:

```
x <- 42.5
y <- 7L
z <- "abc"
typeof(x)
class(x)
is.numeric(x)
typeof(y)
class(y)
is.numeric(y)
typeof(z)
class(z)
is.numeric(z)
```

Output:

```
[1] "double"
[1] "numeric"
[1] TRUE
[1] "integer"
[1] "integer"
[1] TRUE
[1] "character"
[1] "character"
[1] FALSE
```

5. Write an R program to convert a numeric variable into a character variable and a character variable into a numeric variable. Explain the conversion process.

Conversion between types is called type casting. as.character() converts a value to text, and as.numeric() converts text to a number. If the text does not look like a number, as.numeric() returns NA with a warning.

Code:

```
num <- 123.45
char_value <- as.character(num)
print(char_value)
class(char_value)
txt <- "456.78"
num_value <- as.numeric(txt)
print(num_value)
class(num_value)
as.numeric("abc")
```

Output:

```
[1] "123.45"
[1] "character"
[1] 456.78
[1] "numeric"
[1] NA
Warning message:
NAs introduced by coercion
```

6. Consider the following R statements. Identify the data type of each variable and justify your answer.

Code:

```
a <- 25L
b <- 25.5
c <- "R Programming"
d <- FALSE
e <- 3 + 4i
class(a)
class(b)
class(c)
class(d)
class(e)
```

Output:

```
[1] "integer"
[1] "numeric"
[1] "character"
[1] "logical"
[1] "complex"
```

7. Explain type coercion in R with a suitable example involving numeric, character, and logical values.

Type coercion is the automatic conversion of values to a common type. A vector can hold only one data type, so when values of different types are combined, R converts them all to the most flexible type. The order of coercion is: logical < integer < numeric < complex < character. In arithmetic, TRUE becomes 1 and FALSE becomes 0.

Code:

```
v1 <- c(10, "R", TRUE)
print(v1)
class(v1)
v2 <- c(1, TRUE, FALSE)
print(v2)
class(v2)
v3 <- c(TRUE, FALSE)
sum(v3)
TRUE + 5
```

Output:

```
[1] "10"   "R"    "TRUE"
[1] "character"
[1] 1 1 0
[1] "numeric"
[1] 1
[1] 6
```

Part B: Vectors in R

8. Define a vector in R. Explain how to create numeric, character, and logical vectors using the c() function.

A vector is the most basic data structure in R. It is an ordered collection of elements that all have the same data type. The c() function (combine) joins values together to create a vector.

Code:

```
num_vec <- c(10, 20, 30, 40, 50)
char_vec <- c("apple", "banana", "cherry")
logical_vec <- c(TRUE, FALSE, TRUE, TRUE)
print(num_vec)
print(char_vec)
print(logical_vec)
class(num_vec)
class(char_vec)
class(logical_vec)
```

Output:

```
[1] 10 20 30 40 50
[1] "apple"  "banana" "cherry"
[1]  TRUE FALSE  TRUE  TRUE
[1] "numeric"
[1] "character"
[1] "logical"
```

9. Write an R program to create a numeric vector containing ten elements. Perform addition, subtraction, multiplication, and division on the vector.

Code:

```
v <- c(5, 10, 15, 20, 25, 30, 35, 40, 45, 50)
print(v)
add <- v + 5
sub <- v - 3
mul <- v * 2
div <- v / 5
print(add)
print(sub)
print(mul)
print(div)
```

Output:

```
[1]  5 10 15 20 25 30 35 40 45 50
[1] 10 15 20 25 30 35 40 45 50 55
[1]  2  7 12 17 22 27 32 37 42 47
[1] 10 20 30 40 50 60 70 80 90 100
[1]  1  2  3  4  5  6  7  8  9 10
```

10. Explain vector indexing in R with examples of accessing individual elements, selecting multiple elements, and excluding elements using negative indices.

Indexing means accessing elements of a vector by their position. R indexing starts at 1. A single position gives one element, a vector of positions or a range gives several elements, and a negative index removes the element at that position.

Code:

```
v <- c(10, 20, 30, 40, 50)
v[2]
v[c(1, 3, 5)]
v[2:4]
v[-1]
v[-c(2, 4)]
```

Output:

```
[1] 20
[1] 10 30 50
[1] 20 30 40
[1] 20 30 40 50
[1] 10 30 50
```

11. Write an R program to create a vector of student marks and perform the following operations: find the sum and average, identify the maximum and minimum marks, and display marks greater than 75.

Code:

```
marks <- c(78, 85, 62, 91, 70, 88)
cat("Sum:", sum(marks), "\n")
cat("Average:", mean(marks), "\n")
cat("Maximum:", max(marks), "\n")
cat("Minimum:", min(marks), "\n")
cat("Marks greater than 75:", marks[marks > 75], "\n")
```

Output:

```
Sum: 474
Average: 79
Maximum: 91
Minimum: 62
Marks greater than 75: 78 85 91 88
```

12. Explain vector recycling in R. Demonstrate how arithmetic operations work when two vectors have different lengths.

When an operation is performed on two vectors of different lengths, R repeats (recycles) the shorter vector until it matches the length of the longer one. If the longer length is not a multiple of the shorter length, R still recycles but gives a warning.

Code:

```
a <- c(1, 2, 3, 4, 5, 6)
b <- c(10, 20)
print(a + b)
c <- c(1, 2, 3, 4, 5)
d <- c(10, 20)
print(c + d)
```

Output:

```
[1] 11 22 13 24 15 26
```

[1] 11 22 13 24 15

13. Write an R program to create a character vector containing five student names. Sort the names alphabetically, find its length, and access the third element.

Code:

```
students <- c("Rahul", "Anita", "Karthik", "Divya", "Sneha")
sorted_names <- sort(students)
print(sorted_names)
length(students)
students[3]
```

Output:

[1] "Anita" "Divya" "Karthik" "Rahul" "Sneha"
[1] 5
[1] "Karthik"

14. Given the following vectors, determine the output of each operation and explain the result.

Code:

```
x <- c(10, 20, 30, 40, 50)
y <- c(2, 4, 6, 8, 10)
x + y
x * 2
x[c(1, 3, 5)]
x[x > 25]
```

Output:

[1] 12 24 36 48 60
[1] 20 40 60 80 100
[1] 10 30 50
[1] 30 40 50

- **x + y:** element-wise addition, so 10+2, 20+4, 30+6, 40+8 and 50+10 give 12, 24, 36, 48, 60.
- **x * 2:** the single value 2 is recycled and multiplies every element of x.
- **x[c(1, 3, 5)]:** selects the 1st, 3rd and 5th elements, which are 10, 30 and 50.
- **x[x > 25]:** x > 25 gives FALSE FALSE TRUE TRUE TRUE, and logical indexing keeps only the TRUE positions, giving 30, 40, 50.

15. A college stores the attendance percentages of six students in a vector. Write an R program to identify students with attendance above 85%, calculate the average attendance, and count the number of students meeting the attendance requirement.

Code:

```
attendance <- c(92, 78, 88, 95, 81, 86)
above_85 <- attendance[attendance > 85]
print(above_85)
which(attendance > 85)
cat("Average attendance:", mean(attendance), "\n")
cat("Number of students above 85%:", length(above_85), "\n")
```

Output:

[1] 92 88 95 86
[1] 1 3 4 6
Average attendance: 86.66667
Number of students above 85%: 4